

Introduction

In this project, we set out to extend the Snowcast project to utilize gRPC instead of TCP protocol for communication between clients and the server. We also implement adaptive bitrate streaming, where the server will support multiple qualities of bitrate levels. Clients are able to automatically switch between the byte streams to adjust to local network congestion.

Design/Implementation

gRPC Protocol Design

To implement the gRPC protocol, we first define our new messages in a .proto file. All previous message types, such as the HelloMessage, SetStationMessage, WelcomeMessage, InvalidCommandMessage etc. are created as new message headers. Importantly, we note that in Snowcast, we want the connection between the client and server to remain open, so that the server is continually able to send Announce messages and InvalidCommand messages to the client. Because of this, we wrap the AnnounceMessage and InvalidCommand messages in a wrapper ServerEvent Message, which uses the gRPC oneof operator to allow the server to maintain an open stream of either Announce or Invalid command messages to the client.

We also add the function headers for Handshake, SetStation, and Disconnect RPCs inside the .proto file:

```
service SnowcastControl {  
    rpc Handshake(HelloMessage) returns (WelcomeMessage);  
    rpc SetStation(SetStationMessage) returns (stream ServerEvent);  
    rpc Disconnect(DisconnectRequest) returns (DisconnectResponse);  
}
```

After setting up the .proto file, we call the protobuf compiler to compile into protocol_grpc.pb.go, which handles the networking, and protocol.pb.go, which implements the message structs in Go. For ease of use, we also added a setup.sh script which downloads all additional dependencies not present in the original project.

After compiling the protobuf file, we then replace the SnowcastServer struct implementation with a new version that implements UnimplementedSnowcastControlServer from protocol_grpc.pb.go. We chose to implement this class definition so that we could only implement the methods that we care about, and not the other default methods given to us by SnowcastControlServer in protocol_grpc.pb.go.

We then implement the three RPC functions outlined in the protobuf file: Handshake, SetStation, and Disconnect on the newly created SnowcastServer object. Importantly, because we are moving away from TCP connections, we also update the currentClients data structure inside the server to map from a key of “<client IP>:<client port>” to clientInfo struct instead of from conn object to clientInfo struct.

ListStations RPC

The original Snowcast had no way for clients to query what stations were available on the server. Beforehand, the number of stations was communicated via the Welcome message, but the client needed prior knowledge about the station indices in advance. Therefore, for the sake of better user experience, we added a ListStations RPC that returns all station names along with their supported bitrate levels. This feature can be accessed through the client CLI, where typing “l” calls this RPC and then prints the following example result:

```
Station 0: music/song1.mp3 [bitrates: low, medium, high]
Station 1: music/song2.mp3 [bitrates: low, medium, high]
...
```

Adaptive Bitrate Streaming

This is our primary new feature beyond switching to gRPC protocol. Originally, Snowcast streamed audio at a fixed chunk size of ~1500 bytes per tick to all of the clients. Ultimately, we want to replicate a streaming platform that is realistic and efficient for a variety of clients. Therefore, our system supports three quality tiers with the server sending different amounts of data per tick based on each client’s current bitrate. The table below displays the different settings:

Bitrate Level	Chunk Size	Approx. Rate
low	375 bytes	~32 KB/s
medium	750 bytes	~64 KB/s
high	1500 bytes	~128 KB/s

The tick interval that we had still stays constant at ~91.5 ms. Each client will independently get a slice of the buffer sized depending on their chosen bitrate, meaning that multiple clients on the same station at different qualities will receive different amounts of data per tick.

We have two ways that users can switch between different bitrate levels.

Manual selection allows the client to switch between bitrate levels through the client CLI that accepts an optional bitrate arg alongside choosing the station number. An example of this on the

client side would be typing “0 low,” which connects the client to station 0 at low quality. Typing just “0” without the optional bitrate arg defaults the client to high quality.

Automatic switching works via a background goroutine in the client (autoSwitch), which opens up a UDP socket on the listener port and monitors the incoming bytes. Every 3 seconds, the goroutine computes throughput in KB/s and automatic switching follows this logic:

Observed Throughput	Current Bitrate	Action Logic
< 3.0 KB/s	medium or high	Switches to low
3.0 - 10.0 KB/s	high	Switches to medium
>= 3.5 KB/s	low	Switches to medium
>= 7.5 KB/s	medium	Switches to high

When autoSwitch decides to change the bitrate, it calls SetStation with the new bitrate value, which is the same RPC that the user calls manually. We also have a flushMetrics flag that discards one measurement window after any type of switch, and this is done so that we can prevent stale data from the previous bitrate tier from triggering an immediate follow-up switch.

We additionally have metric logging on the client-side where both the controller and listener report the stream metrics to stderr. The controller reports the throughput and current bitrate every 3 secs, while the listener reports throughput, average chunk arrival interval, and buffer health (which gets flagged as “degraded” if arrival interval deviates significantly from our expected ~91.5 ms tick).

Results

New Testing Infrastructure

Because the gRPC implementation uses a different networking protocol than the original Snowcast project, the original tests that came with the project are no longer compatible. To fix this, we wrote several unit tests checking the correctness of each RPC function, as well as the handshake. To make running these tests easier, we also added an additional shell script, run-tests.sh. Our implementation passes all unit tests:

```
Snowcast server started on port 16800 with 7 stations
=== RUN    TestHandshake
Client handshake received on UDP port 5000
Client successfully connected: 127.0.0.1:5000
--- PASS: TestHandshake (0.00s)
=== RUN    TestSetStationValid
Client handshake received on UDP port 5001
Client successfully connected: 127.0.0.1:5001
--- PASS: TestSetStationValid (0.00s)
=== RUN    TestSetStationInvalid
Client handshake received on UDP port 5002
Client successfully connected: 127.0.0.1:5002
--- PASS: TestSetStationInvalid (0.00s)
=== RUN    TestDisconnect
Client handshake received on UDP port 5003
Client successfully connected: 127.0.0.1:5003
Client disconnected: 127.0.0.1:5003
--- PASS: TestDisconnect (0.00s)
PASS
ok          snowcast-jamesyan2028/tests    0.018s
```

Adaptive Bitrate Streaming Results

We also tested the auto-switching using the tc command for traffic control on the loopback interface. We used the following command: “sudo tc qdisc add dev lo root tbf rate 16kbit burst 32kbit latency 400ms”.

Under the restricted network conditions, the client was able to detect the under 3KB/s throughput within one measurement window and automatically calls SetStation to switch to the lower bitrate accordingly. When we removed the tc restriction, our throughput recovered and the client was able to step the bitrate level back up to medium and then to high over two windows (see demo video for example). Regarding our design choice, we decided to have this 3 second window because even though the client is always slightly behind the true network condition, this design prevents the client from rapidly changing between tiers in the case of fluctuations in network traffic. This was a challenge that we had previously where the bitrate would switch pretty rapidly.

AI Usage

AI was used to gain an understanding of how to use gRPC and its interface semantics. Claude was very thorough in its explanation of gRPC concepts, and explained how to use the protobuf compiler and how to use the generated boilerplate files effectively. We did not stray in our AI usage from our proposal.

Conclusion/Future Work

In working on this project, we gained experience in constructing protocols using gRPC, which we found to be much easier to use than manually writing and deserializing bytes in TCP sockets. The adaptive streaming feature that we added was also quite satisfying because seeing the client respond to simulated network degradation and recovery made our platform feel functional in a realistic setting. If we could continue working on this project, we could extend the backend to

make it fault tolerant using a second server as a backup, and scale out the backend with load balancing. We could additionally look into smarter and more efficient algorithms for adaptive bitrate.